4-DOF Robotic Arm: a Virtual Laboratory

Literature review and possible directions

Project team

Hasan Ata Yılmaz	040240789	yilmazhas24@itu.edu.tr
Yusuf Taha Okutan	040200632	okutan20@itu.edu.tr
Eren Taşkın	040220768	taskine21@itu.edu.tr
Yavuz Selim Al	040220759	al22@itu.edu.tr
Muhammed Alper Demetçiçek	040200544	demetcicek20@itu.edu.tr

October 2026

In short

- **The idea.** A virtual laboratory, written in C# on .NET 10 with a Windows Forms panel, for a robotic arm with four joints. The course builds one for magnetic levitation; we want to apply the same structure to a different plant.
- **Where we stand.** The project is at an early stage. This report surveys the arms that exist in industry and in teaching laboratories, the published work on modelling, controlling and simulating them, and sketches a direction the project can grow along. It is not a fixed specification.
- **Current inclination.** An open-chain arm with a base yaw joint and three pitch joints, with its dimensions and masses taken from a real educational arm, the OpenMANIPULATOR-X, so that the model does not rest on guessed numbers.

1 Introduction

A virtual laboratory is a program that stands in for a physical experiment: a model of the plant, a controller, an interface a person can operate, and the recording machinery of a real rig (Lecture Note 1). The course organises every control application into six parts, and Table 1 shows what each part would be for a robotic arm.

Part	For a robotic arm
Plant model	Rigid-body dynamics of the arm: four joint angles and four joint speeds as state, four joint torques as input
Solver	A fixed-step integrator such as RK4
Controller	Joint-level PID to begin with, model-based control later
Hardware I/O	Sensors and motors, emulated in software unless real hardware becomes available
Storage	Experiment logs that can be reloaded and compared
User interface	A panel that draws the arm, takes targets and plots angles, errors and torques

Table 1. The six parts of a control application (Lecture Note 1) for a robotic arm. The first three form the computational core.

Three things make the arm a different problem from the levitation rig. It has **several inputs**, one torque per joint, where the course interfaces carry a single input. Its joints are **coupled**: moving one changes the load on the others, so every evaluation of the model involves a 4×4 inertia matrix. And while the levitation rig is unstable with a single real pole, an arm left without control behaves like a **multi-link pendulum**, which is chaotic. That changes which checks a simulation can be trusted with.

2 Literature review

We looked at three kinds of source: robot arms that exist in industry and in teaching laboratories, papers on modelling and controlling arms with four joints, and software used to simulate and teach manipulators.

2.1 Four-axis arms in industry and in teaching

Four axes is a common count in industry because many tasks need only a position and one rotation of the tool. But "four axes" covers machines that are built very differently, and the differences decide how such an arm would be modelled and controlled. Table 2 lists the main families and representative products.

2.2 Our choice of arm, and why

At this stage we lean towards an open-chain arm with a base yaw joint and three pitch joints, modelled on the OpenMANIPULATOR-X, for three reasons.

1. **It keeps the whole problem.** It is the only layout in Table 2 in which all four joints are actuated, form an open chain, carry gravity load on three of them and set the pitch of the tool. The full coupled model then matters, and that is what makes controller comparisons and correctness checks meaningful.
2. **Its inverse kinematics has a closed form** (Section 2.3), so the simulator needs no iterative solver to reach a target.
3. **A real, documented example exists.** In the OpenMANIPULATOR-X model, joint 1 turns about the vertical axis and joints 2 to 4 about parallel horizontal axes. The second link is about 0.13 m long with a 24 mm offset, the third about 0.124 m, and the tool point lies about 0.126 m beyond the wrist [11]. The manufacturer's table lists four moving bodies of about 105, 142, 135 and 236 g with their inertia tensors [10], and the actuators have a 353.5:1 gearbox and 4.1 N m stall torque at 12 V [12]. Building the model from these numbers makes it realistic, and it keeps later steps open: actuator limits, and real hardware if it ever becomes available.


SCARA (FANUC SR-3iA)


ABB IRB 460


ABB IRB 360 FlexPicker


Dobot Magician

Arm	Structure	Typical use	What it means for a simulation project
SCARA [1, 2]	Usually four axes: two rotary joints moving the arm in a horizontal plane, plus vertical motion and tool rotation	Fast assembly, pick-and-place, electronics	Gravity does not load the horizontal joints, so much of the interesting dynamics is absent
Four-axis palletiser, e.g. ABB IRB 460 [3]	Serial arm whose parallelogram linkages keep the tool horizontal [4]	Palletising; the IRB 460 carries 110 kg over a 2.4 m reach at about 2 000 cycles per hour	Closed kinematic chains; the tool's pitch is fixed by the mechanism, not controlled
Delta robot, e.g. ABB IRB 360 FlexPicker [5]	Parallel arms, three or four axes	High-speed picking and packing, 1 to 8 kg	Parallel kinematics, a different modelling problem altogether
Dobot Magician [6, 7]	Four controllable joints and a passive joint that keeps the tool holder horizontal	Teaching and desktop automation; 0.5 kg, 320 mm reach	The same limitation as the palletisers
Quanser QArm [8]	Serial, four joints: roll, pitch, pitch, roll	Control education and research; MATLAB/Simulink, Python and ROS, with a virtual twin	The closest existing product to our idea; the last joint rolls the tool, so its pitch is not controlled
Arduino Braccio [9]	Six servo-driven axes	Hobby and teaching; 150 g at 32 cm	Not a four-joint arm; hobby servos, and we found no published link inertia data
OpenMANIPULATOR-X [10, 11]	Serial: base yaw and three pitch joints, plus a gripper; DYNAMIXEL XM430-W350 actuators	Teaching and research with ROS; 500 g payload, 380 mm reach	All four joints actuated in an open chain; the manufacturer publishes masses, centres of gravity, inertia tensors and a URDF model

Table 2. Four-axis arms in industry and in teaching laboratories.


Quanser QArm


Arduino Braccio


OpenMANIPULATOR-X

The arms of Table 2. Product photographs in table order: four above the table, three below. Images are taken from the manufacturers' web pages; the SCARA shown is a FANUC SR-3iA.

2.3 Kinematics

The standard description of a serial arm is the Denavit–Hartenberg convention [13], covered in the main robotics textbooks [14, 15, 16]. Mohammed and Sunar [17] model a 4-DOF arm with both Denavit–Hartenberg parameters and the product-of-exponentials formula, find that the two give identical forward kinematics, and solve the inverse problem algebraically. Bao et al. [18] study the layout we lean towards, a waist joint followed by three joints with parallel axes. They obtain the first angle from the target position, the next two from the triangle formed by the links (law of cosines) and the last from the required tool orientation, and note that a pose can have up to four solutions. For a Braccio arm, where this shortcut is not available, Bogdanovskyi et al. [19] use Levenberg–Marquardt iterations with an analytic Jacobian.

2.4 Dynamics, and how others checked their models

Most 4-DOF studies use the Euler–Lagrange method, which gives the closed form $\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \boldsymbol{\tau}$. Al-Qahtani et al. [20] derive it for a 4-link arm from the body Jacobian of each link and analyse the properties of the model that matter for control. Li et al. [21] build both a Lagrange and a Kane model of a 4-DOF modular robot and compare them in simulation. Adar et al. [22] derive Lagrange–Euler equations for a 5-DOF arm and, before designing a controller, let the links fall freely to see whether the equations of motion behave physically.

The recursive Newton–Euler algorithm of Luh, Walker and Paul [23] computes joint torques with a cost proportional to the number of joints, and Walker and Orin [24] showed how to use it to simulate forward dynamics; Featherstone [25] is the standard reference. Textbooks list the structural properties a correct model must have: $\mathbf{M}$ is symmetric positive definite, and $\dot{\mathbf{M}} - 2\mathbf{C}$ is skew-symmetric when $\mathbf{C}$ is built from the Christoffel symbols [14]. These properties, and the free-fall idea of [22], can be turned directly into automated tests.

2.5 Control

Three controllers recur. Independent-joint PID treats each joint as a separate loop and the coupling as a disturbance; Mata-Machuca et al. [26] report acceptable trajectory tracking with it on a 4-DOF arm in simulation, and Bonikila et al. [27] use PID with inverse kinematics to follow straight and parabolic paths. PD control with gravity compensation is the simplest law with a stability proof for reaching a fixed target [28, 29]. Computed torque control, also called feedback linearisation, uses the model to cancel the nonlinear terms: Al-Qahtani et al. [20] apply it alongside a PID tuned by differential evolution, Vidaković et al. [30] report a clear tracking improvement over a PI speed loop on a 4-DOF device, and Shah and Rattan [31] combine it with PID on a two-link arm.

Computed torque is only as good as the model it uses. A comparison between PID and computed torque that includes a deliberately wrong model, for example a wrong payload mass, is therefore a natural experiment for a virtual laboratory.

2.6 Simulation software and virtual laboratories

Heradio et al. [32] survey virtual and remote laboratories in control education: what they offer, which ones exist and how they are deployed. For manipulators, the established teaching tools are libraries inside a numerical environment or stand-alone programs. ARTE [33] is a MATLAB toolbox for teaching manipulators, with realistic 3D visualisation and robot programming; RoboAnalyzer [34] is a stand-alone program built around 3D models and Denavit–Hartenberg parameters; the Robotics Toolbox now also exists for Python [35]. General simulators such as V-REP [36] sit at the other end, with a physics engine that integrates the motion for the user. Two reviews [37, 38] catalogue further educational simulators. On the product side, the Quanser QArm comes with a virtual twin for courses taught at a distance [8]. The closest work to our technology is a Windows

Forms simulator for a Braccio arm [19], which covers kinematics only.

2.7 What this suggests

> Two observations shape our direction. First, in the studies we reviewed, the tool is usually MATLAB, and the evidence that a simulation is right is mostly a tracking plot, an agreement between two derivations or a check on a few poses; we found no study that presents the simulator itself as software with an architecture and automated tests. Second, the closest educational product, the QArm with its virtual twin, is commercial and tied to its own environment.
>
> A small, open, tested virtual laboratory in C# for a documented arm is therefore a reasonable contribution for a course project. Its value would lie less in a new controller than in showing that the simulation can be trusted.

3 A first model of the arm

This section writes down the model a first version would implement: the geometry of the OpenMANIPULATOR-X, its forward and inverse kinematics, its Jacobian, and its equations of motion in the form a fixed-step solver needs. Angles and dimensions follow the manufacturer's URDF file [11], so that an angle in the simulator means the same thing on the real arm and in its ROS software. Each closed-form expression below was checked numerically against the transform chain of that file.

3.1 Coordinates and geometry

The base frame has its origin on the axis of joint 1, with z pointing up and x pointing forward when $q_1 = 0$; in the URDF this axis lies 12 mm in front of the base origin. Joint 1 turns the arm about z, and joints 2, 3 and 4 turn about horizontal axes perpendicular to the vertical plane that contains the arm. As in the URDF, a positive pitch angle tips the arm forward and down. Because the three pitch axes are parallel, their angles add up, and it is convenient to work with the absolute pitch of each body below the horizontal (Figure 1):

$$\theta_2 = q_2, \qquad \theta_3 = q_2 + q_3, \qquad \theta_4 = q_2 + q_3 + q_4. \tag{1}$$

Body i is the part that moves with joint i; body 4 includes the gripper. Joint 3 sits 24 mm forward of and 128 mm above joint 2 when $q_2 = 0$, so link 2 acts as a straight link of length L_2 whose elevation, seen from joint 2, is β at $q_2 = 0$ and $\beta - \theta_2$ in general. Table 3 lists the lengths.

3.2 Forward kinematics

In the vertical plane of the arm, the tool point lies at horizontal distance r from the joint-1 axis and at height z:

$$\begin{aligned} r &= L_2 \cos(\theta_2 - \beta) + L_3 \cos\theta_3 + L_4 \cos\theta_4, \\ z &= d_1 - L_2 \sin(\theta_2 - \beta) - L_3 \sin\theta_3 - L_4 \sin\theta_4. \end{aligned} \tag{2}$$

Joint 1 turns this plane about the vertical axis, which gives the tool position $\mathbf{p}$ and the pitch ψ of the tool below the horizontal:

$$\mathbf{p} = \begin{bmatrix} r\cos q_1 & r\sin q_1 & z \end{bmatrix}^{\mathsf{T}}, \qquad \psi = \theta_4. \tag{3}$$

The same chain in Denavit–Hartenberg form [13, 14] has the parameters of Table 4, with

$$\mathbf{A}_i = \mathrm{Rot}_z(\vartheta_i)\,\mathrm{Trans}_z(d_i)\,\mathrm{Trans}_x(a_i)\,\mathrm{Rot}_x(\alpha_i), \qquad \mathbf{T}_4^0(\mathbf{q}) = \mathbf{A}_1\mathbf{A}_2\mathbf{A}_3\mathbf{A}_4. \tag{4}$$

Symbol	Meaning	Value
d_1	Height of the joint-2 axis above the joint-1 frame	59.5 mm
L_2	Joint 2 to joint 3 (24 mm forward, 128 mm up)	130.2 mm
β	Elevation of joint 3 seen from joint 2 at $q_2 = 0$, atan2(128, 24)	79.38° (1.3854 rad)
L_3	Joint 3 to joint 4	124 mm
L_4	Joint 4 to the tool point	126 mm

Table 3. Geometric parameters of the OpenMANIPULATOR-X, from its URDF file [11].

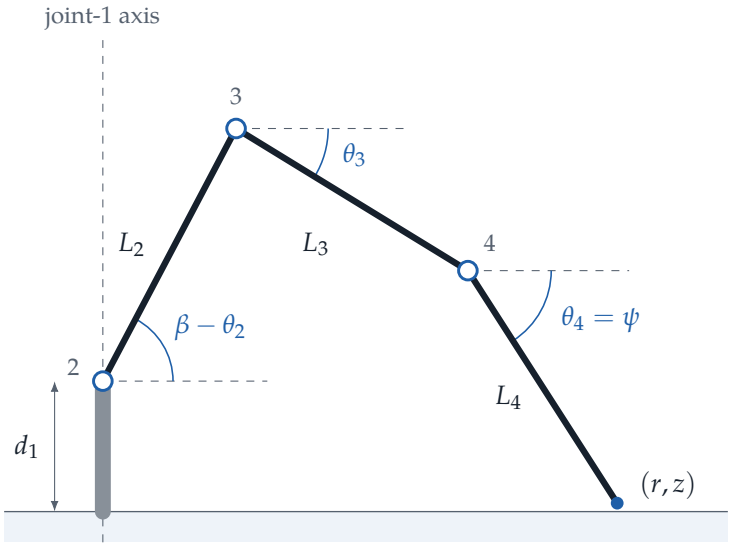


Figure 1. Side view of the arm in its vertical plane, drawn to scale at $q_2 = 0.30$, $q_3 = 0.25$, $q_4 = 0.45$ rad. Pitch angles are measured below the horizontal; the grey bar is the base and (r, z) the tool point.

The position column of $\mathbf{T}_4^0$ reproduces (2)–(3). The matrix form is the one to use for drawing the arm, since the partial products also give every intermediate frame.

3.3 Inverse kinematics

Given a target position $\mathbf{p} = (x, y, z)$ and a tool pitch ψ, joint 1 follows from the direction of the target and the other three from a two-link problem in the vertical plane, as in [18]. The wrist (joint 4) must lie a distance L_4 back from the target along the tool, at (u, v) relative to joint 2:

$$q_1 = \operatorname{atan2}(y, x), \qquad u = \sqrt{x^2 + y^2} - L_4 \cos\psi, \qquad v = z - d_1 + L_4 \sin\psi. \tag{5}$$

The law of cosines gives the angle φ from link 2 to link 3, and from it the elevation e of link 2 above the horizontal:

$$\cos\varphi = \frac{u^2 + v^2 - L_2^2 - L_3^2}{2 L_2 L_3}, \qquad e = \operatorname{atan2}(v, u) - \operatorname{atan2}\big(L_3 \sin\varphi,\ L_2 + L_3 \cos\varphi\big), \tag{6}$$

$$q_2 = \beta - e, \qquad q_3 = -\beta - \varphi, \qquad q_4 = \psi - q_2 - q_3. \tag{7}$$

The target is reachable only if $|\cos\varphi| \le 1$. Taking $\varphi = -\arccos(\cdot)$ gives the elbow-up pose of Figure 1 and $+\arccos(\cdot)$ the elbow-down pose. Turning the base by π and reaching back over it ($q_1 + \pi$, with ψ replaced by $\pi - \psi$ and u computed from $-\sqrt{x^2 + y^2}$) gives the other two of the four solutions noted in [18]; the joint limits usually exclude them.

Link i	ϑ_i	d_i	a_i	α_i
1	q_1	d_1	0	$-\pi/2$
2	$q_2 - \beta$	0	L_2	0
3	$q_3 + \beta$	0	L_3	0
4	q_4	0	L_4	0

Table 4. Denavit–Hartenberg parameters of the arm with the joint angles of the URDF file. Frame 4 sits at the tool point with its x axis along the tool.

3.4 Velocity kinematics

Differentiating (2) gives the Jacobian of the planar coordinates (r, z, ψ) with respect to the pitch joints,

$$\begin{bmatrix} \dot r \\ \dot z \\ \dot\psi \end{bmatrix} = \mathbf{J}_p(\mathbf{q}) \begin{bmatrix} \dot q_2 \\ \dot q_3 \\ \dot q_4 \end{bmatrix}, \qquad \mathbf{J}_p = \begin{bmatrix} -(s_2+s_3+s_4) & -(s_3+s_4) & -s_4 \\ -(c_2+c_3+c_4) & -(c_3+c_4) & -c_4 \\ 1 & 1 & 1 \end{bmatrix}, \tag{8}$$

with $s_2 = L_2\sin(\theta_2 - \beta)$, $c_2 = L_2\cos(\theta_2 - \beta)$ and $s_i = L_i \sin\theta_i$, $c_i = L_i\cos\theta_i$ for $i = 3, 4$; joint 1 adds a tangential speed $r\dot q_1$. The determinant

$$\det \mathbf{J}_p = -L_2 L_3 \sin(q_3 + \beta) \tag{9}$$

vanishes when links 2 and 3 are in line ($q_3 = -\beta$ with the arm stretched, $q_3 = \pi - \beta$ with it folded). These poses, and $r = 0$, where joint 1 no longer moves the tool point, are the singular configurations. Away from them, straight-line moves of the tool can be computed from $\dot{\mathbf{q}} = \mathbf{J}^{-1}\dot{\mathbf{x}}$.

3.5 Equations of motion

With the Lagrangian $\mathcal{L} = T - V$, where

$$T = \tfrac{1}{2}\dot{\mathbf{q}}^{\mathsf T}\mathbf{M}(\mathbf{q})\dot{\mathbf{q}}, \qquad V = g\sum_{i=1}^{4} m_i\, z_{c,i}(\mathbf{q}), \tag{10}$$

the Euler–Lagrange equations $\frac{\mathrm d}{\mathrm dt}\frac{\partial \mathcal L}{\partial \dot{\mathbf q}} - \frac{\partial \mathcal L}{\partial \mathbf q} = \boldsymbol\tau - \boldsymbol\tau_f$ give

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \boldsymbol\tau - \boldsymbol\tau_f. \tag{11}$$

The inertia matrix sums the contributions of the four bodies,

$$\mathbf{M}(\mathbf{q}) = \sum_{i=1}^{4}\Big(m_i\, \mathbf{J}_{v,i}^{\mathsf T}\mathbf{J}_{v,i} + \mathbf{J}_{\omega,i}^{\mathsf T}\mathbf{R}_i\mathbf{I}_i\mathbf{R}_i^{\mathsf T}\mathbf{J}_{\omega,i}\Big), \tag{12}$$

where m_i is the mass of body i, $\mathbf{I}_i$ its inertia tensor about its centre of mass in its own frame, $\mathbf{R}_i$ the orientation of that frame, $z_{c,i}$ the height of its centre of mass, and $\mathbf{J}_{v,i}$, $\mathbf{J}_{\omega,i}$ the Jacobians of the velocity of its centre of mass and of its angular velocity. The Coriolis and centrifugal terms follow from $\mathbf{M}$ through the Christoffel symbols, and the gravity terms from V:

$$C_{kj} = \sum_{i=1}^{4} c_{ijk}\,\dot q_i, \qquad c_{ijk} = \frac{1}{2}\left(\frac{\partial M_{kj}}{\partial q_i} + \frac{\partial M_{ki}}{\partial q_j} - \frac{\partial M_{ij}}{\partial q_k}\right), \qquad g_k = \frac{\partial V}{\partial q_k}. \tag{13}$$

Building $\mathbf{C}$ this way, rather than by another split of the same terms, is what makes $\dot{\mathbf{M}} - 2\mathbf{C}$ skew-symmetric (Section 3.7).

The inertial data come from the URDF file [11], which gives the centres of mass and inertia tensors in the same joint frames as the geometry (Table 5). The specification table [10] lists different masses (105 to 236 g, against 98 to 143 g in the URDF), so the simulator should take all of them from one source.

Body	m_i (g)	ℓ_i (mm)	ε_i	$I_{xx,i}$	$I_{yy,i}$	$I_{zz,i}$	$I_{xz,i}$
1 (turret)	98.4	47.4	$-90.4°$	34.5	32.7	18.9	-0.4
2	138.5	102.2	$-84.2°$	330.6	342.9	60.3	-38.5
3	132.7	90.9	$-0.1°$	30.7	242.3	251.6	-0.3
4 (with gripper)	143.3	45.1	$-11.4°$	80.9	76.0	93.1	-1.0

Table 5. Inertial parameters of the moving bodies from the URDF file [11]. The centre of mass of body i lies at distance ℓ_i from joint i, at the angle $\theta_i + \varepsilon_i$ below the horizontal ($\theta_1 = 0$). Inertias are about the centre of mass, in 10^{-6} kg m^2.

Structure of the model for this arm. The centres of mass lie within 0.6 mm of the vertical plane of the arm, and the products of inertia I_{xy} and I_{yz} are small. Neglecting them, $\mathbf{M}$ splits into a yaw entry m_{11} and a 3×3 pitch block, and every term can be written in closed form. The yaw entry is

$$m_{11} = \sum_{i=1}^{4} \Big(m_i \rho_i^2 + I_{xx,i} \sin^2\theta_i + I_{zz,i} \cos^2\theta_i - 2 I_{xz,i} \sin\theta_i \cos\theta_i \Big), \tag{14}$$

where ρ_i is the horizontal distance of the centre of mass of body i from the joint-1 axis:

$$\begin{gathered} \rho_1 = \ell_1 \cos\varepsilon_1, \qquad \rho_2 = \ell_2 \cos(\theta_2 + \varepsilon_2), \qquad \rho_3 = L_2 \cos(\theta_2 - \beta) + \ell_3 \cos(\theta_3 + \varepsilon_3), \\ \rho_4 = L_2 \cos(\theta_2 - \beta) + L_3 \cos\theta_3 + \ell_4 \cos(\theta_4 + \varepsilon_4). \end{gathered} \tag{15}$$

The pitch block depends only on q_3 and q_4. With $J_i = I_{yy,i}$ and the coupling terms

$$\begin{aligned} h &= L_2 \ell_3 \cos(q_3 + \beta + \varepsilon_3), & k_{23} &= L_2 L_3 \cos(q_3 + \beta), \\ k_{24} &= L_2 \ell_4 \cos(q_3 + q_4 + \beta + \varepsilon_4), & k_{34} &= L_3 \ell_4 \cos(q_4 + \varepsilon_4), \end{aligned} \tag{16}$$

its entries are, with $M_{jk} = M_{kj}$,

$$\begin{aligned} M_{44} &= J_4 + m_4 \ell_4^2, \\ M_{34} &= J_4 + m_4 (\ell_4^2 + k_{34}), \\ M_{33} &= J_3 + J_4 + m_3 \ell_3^2 + m_4 (L_3^2 + \ell_4^2 + 2k_{34}), \\ M_{24} &= J_4 + m_4 (\ell_4^2 + k_{34} + k_{24}), \\ M_{23} &= J_3 + J_4 + m_3 (\ell_3^2 + h) + m_4 (L_3^2 + \ell_4^2 + 2k_{34} + k_{23} + k_{24}), \\ M_{22} &= J_2 + J_3 + J_4 + m_2 \ell_2^2 + m_3 (L_2^2 + \ell_3^2 + 2h) + m_4 (L_2^2 + L_3^2 + \ell_4^2 + 2k_{23} + 2k_{24} + 2k_{34}). \end{aligned} \tag{17}$$

The gravity torques, accumulated from the tool towards the base, are

$$\begin{aligned} g_4 &= -g\, m_4 \ell_4 \cos(\theta_4 + \varepsilon_4), \\ g_3 &= g_4 - g \big[m_3 \ell_3 \cos(\theta_3 + \varepsilon_3) + m_4 L_3 \cos\theta_3 \big], \\ g_2 &= g_3 - g \big[m_2 \ell_2 \cos(\theta_2 + \varepsilon_2) + (m_3 + m_4) L_2 \cos(\theta_2 - \beta) \big], \end{aligned} \tag{18}$$

and $g_1 = 0$, since joint 1 turns about a vertical axis. With this structure, (11) and (13) separate into an equation for the base and three for the pitch joints:

$$\begin{aligned} & m_{11}\, \ddot{q}_1 + \dot{q}_1 \sum_{j=2}^{4} \frac{\partial m_{11}}{\partial q_j}\, \dot{q}_j = \tau_1 - \tau_{f,1}, \\ & \sum_{j=2}^{4} M_{kj}\, \ddot{q}_j + \sum_{i,j=2}^{4} c_{ijk}\, \dot{q}_i \dot{q}_j - \frac{1}{2} \frac{\partial m_{11}}{\partial q_k}\, \dot{q}_1^2 + g_k = \tau_k - \tau_{f,k}, \qquad k = 2, 3, 4. \end{aligned} \tag{19}$$

The last term on the left of the second line is the centrifugal pull of a turning base on the pitch joints, and the sum in the first line is the matching Coriolis torque on joint 1. With the out-of-plane terms removed, (14)–(19) agree with (12)–(13) to rounding error; (18) is exact even with the full data, because the out-of-plane offsets do not change the heights of the centres of mass. With the full tensors the yaw–pitch entries M_{1k} are no longer zero, though they stay below 5 % of m_{11} over random poses. The simulator should therefore evaluate the general expressions (12)–(13), and use the closed forms to understand the model and to test it.

Actuators and friction. Each joint is driven through a 353.5:1 gearbox [12], so a first model treats each actuator as a limited torque source with friction:

$$\tau_i = \mathrm{sat}(u_i, \tau_{\max}), \qquad \tau_{f,i} = f_{v,i}\,\dot{q}_i + f_{c,i}\tanh(\dot{q}_i/\omega_s), \tag{20}$$

where u_i is the controller output, $\tau_{\max} = 4.1$ N m the stall torque at 12 V [12], $f_{v,i}$ and $f_{c,i}$ viscous and Coulomb coefficients, and ω_s a small speed that smooths the sign function so that the fixed-step solver does not chatter at zero speed. The rotor inertia reflected through the gearbox, $N^2 J_m$ with $N = 353.5$, adds to the diagonal of $\mathbf{M}$. The friction coefficients and J_m are parameters to identify or to vary in experiments.

3.6 State-space form and integration

With the state $\mathbf{x} = [\mathbf{q}^\mathsf{T}\ \dot{\mathbf{q}}^\mathsf{T}]^\mathsf{T} \in \mathbb{R}^8$ and the input $\mathbf{u}$, (11) becomes

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}) = \begin{bmatrix} \dot{\mathbf{q}} \\ \mathbf{M}(\mathbf{q})^{-1}\big(\boldsymbol{\tau}(\mathbf{u}) - \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} - \mathbf{g}(\mathbf{q}) - \boldsymbol{\tau}_f(\dot{\mathbf{q}})\big) \end{bmatrix}. \tag{21}$$

Because $\mathbf{M}$ is symmetric positive definite, the accelerations are best found by a Cholesky solve of $\mathbf{M}\ddot{\mathbf{q}} = \mathbf{b}$ rather than by inverting $\mathbf{M}$. The fixed-step RK4 solver advances (21) with step h by

$$\begin{gathered} \mathbf{k}_1 = \mathbf{f}(\mathbf{x}_n, \mathbf{u}_n), \quad \mathbf{k}_2 = \mathbf{f}\big(\mathbf{x}_n + \tfrac{h}{2}\mathbf{k}_1, \mathbf{u}_n\big), \quad \mathbf{k}_3 = \mathbf{f}\big(\mathbf{x}_n + \tfrac{h}{2}\mathbf{k}_2, \mathbf{u}_n\big), \quad \mathbf{k}_4 = \mathbf{f}(\mathbf{x}_n + h\mathbf{k}_3, \mathbf{u}_n), \\ \mathbf{x}_{n+1} = \mathbf{x}_n + \tfrac{h}{6}\big(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4\big), \end{gathered} \tag{22}$$

with the input held over the step, as a digital controller with a zero-order hold would hold it. The global error of the scheme falls as h^4.

3.7 Properties that become tests

The model has properties that hold for every trajectory, and each can be written as an automated test:

1. **Inertia.** $\mathbf{M}(\mathbf{q}) = \mathbf{M}(\mathbf{q})^\mathsf{T}$ and $\mathbf{M}(\mathbf{q}) \succ 0$ in every pose.
2. **Skew symmetry.** $\mathbf{z}^\mathsf{T}\big(\dot{\mathbf{M}} - 2\mathbf{C}\big)\mathbf{z} = 0$ for every $\mathbf{q}$, $\dot{\mathbf{q}}$ and $\mathbf{z}$.
3. **Energy.** $E = \frac{1}{2}\dot{\mathbf{q}}^\mathsf{T}\mathbf{M}\dot{\mathbf{q}} + V$ satisfies $\dot{E} = \dot{\mathbf{q}}^\mathsf{T}(\boldsymbol{\tau} - \boldsymbol{\tau}_f)$, so it stays constant when $\boldsymbol{\tau} = \boldsymbol{\tau}_f = \mathbf{0}$.
4. **Angular momentum about the vertical axis.** Since q_1 does not appear in $\mathcal{L}$, the momentum $p_1 = \sum_j M_{1j}\dot{q}_j$ obeys $\dot{p}_1 = \tau_1 - \tau_{f,1}$ and stays constant when joint 1 is free.
5. **Statics.** The torque $\boldsymbol{\tau} = \mathbf{g}(\mathbf{q})$ holds the arm at rest in any pose.
6. **Kinematics.** (5)–(7) applied to the output of (2)–(3) return the original angles, and $\mathbf{J}_p$ matches a finite-difference derivative of (2).
7. **Convergence.** Halving h divides the error after a fixed time by about $2^4 = 16$.

A short prototype of these checks in Python, run on the URDF data, gives an energy drift of about 2×10^{-6} J and a drift of p_1 below 10^{-8} kg m^2/s over 0.4 s of free motion with $h = 2$ ms, and an error ratio of 15.1 when h is halved: what a correct model and solver should show.

4 Where the project could go

A first version would model the arm with the Euler–Lagrange equations and the published OpenMANIPULATOR-X parameters (Section 3), advance it with a fixed-step RK4 solver, drive it with joint-level PID and show it in a Windows Forms panel. From there the project can grow in several directions, depending on what the course covers and on the time available:

- **Showing that the simulation is right.** Checks that follow from the physics and can run as automated tests (Section 3.7): with no torque and no friction, the energy and the angular momentum about the vertical axis stay constant; halving the step reduces the error by the factor the solver's order predicts; inverse kinematics undoes forward kinematics; the gravity torque alone holds the arm still. Because a free arm is chaotic, long runs are judged by these conserved quantities rather than by comparing trajectories.
- **Controller comparison.** PID, PD with gravity compensation and computed torque on the same moves, including runs with a deliberately wrong model.
- **Motion planning.** Smooth joint trajectories (cubic or quintic polynomials), and straight-line moves of the tool in Cartesian space.
- **Operating modes.** A finite state machine for idle, homing, moving and fault states, with joint limits that stop the arm.
- **More realistic hardware behaviour.** Encoder resolution, torque and speed limits taken from the actuator data sheet, and sampling delay.
- **Data.** Logging, replaying and querying recorded experiments.
- **Visualisation.** A 3D view of the arm in addition to a simple 2D drawing.
- **Real hardware.** If an OpenMANIPULATOR-X becomes available, the same controller code could drive it, which is the hardware-in-the-loop stage of the course.

References

[1] H. Makino, “Development of the SCARA,” *Journal of Robotics and Mechatronics*, vol. 26, no. 1, pp. 5–8, 2014. doi:10.20965/jrm.2014.p0005

[2] RoboDK, “What is a SCARA robot? The background and benefits.” `https://robodk.com/blog/?p=6099`

[3] ABB Robotics, “IRB 460 product presentation.” `https://library.e.abb.com/public/1a92f5e79358dcdcc1257863004af353/IRB%20460%20Product%20presentation.pdf`

[4] Duet3D documentation, “Robot, 4-axis palletized.” `https://docs.duet3d.com/User_manual/Machine_configuration/robot_4_axis_palletized`

[5] ABB Robotics, “IRB 360 FlexPicker.” `https://www.abb.com/global/en/areas/robotics/products/robots/delta-robots/irb-360`

[6] RoboDK robot library, “Dobot Magician.” `https://robodk.com/robot/Dobot/Magician`

[7] Université d’Angers, LARIS, “TP 1: Dobot Magician, présentation du robot et de DobotStudio” (laboratory handout). `https://perso-laris.univ-angers.fr/~boimond/TP_1_Dobot_Magician_Presentation_du_robot_et_de_DobotStudio.pdf`

[8] Quanser, “QArm.” `https://www.quanser.com/products/qarm/`

[9] Arduino, “Tinkerkit Braccio robot.” `https://store-usa.arduino.cc/products/tinkerkit-braccio-robot`

[10] ROBOTIS e-Manual, “OpenMANIPULATOR-X: Specification.” `https://emanual.robotis.com/docs/en/platform/openmanipulator_x/specification/`

[11] ROBOTIS, `open_manipulator` repository, file `open_manipulator_x.urdf`. `https://github.com/ROBOTIS-GIT/open_manipulator`

[12] ROBOTIS e-Manual, “DYNAMIXEL XM430-W350.” `https://emanual.robotis.com/docs/en/dxl/x/xm430-w350`

[13] J. Denavit and R. S. Hartenberg, “A kinematic notation for lower-pair mechanisms based on matrices,” *Journal of Applied Mechanics*, vol. 22, no. 2, pp. 215–221, 1955.

[14] M. W. Spong, S. Hutchinson and M. Vidyasagar, *Robot Modeling and Control*, 2nd ed. Wiley, 2020.

[15] B. Siciliano, L. Sciavicco, L. Villani and G. Oriolo, *Robotics: Modelling, Planning and Control*. Springer, 2009.

[16] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 4th ed. Pearson, 2018.

[17] A. A. Mohammed and M. Sunar, “Kinematics modeling of a 4-DOF robotic arm,” in *Proc. 2015 Int. Conf. on Control, Automation and Robotics* (*ICCAR*), pp. 87–91, 2015.

[18] G. Bao, S. Liu and H. Zhao, “Kinematics simulation of 4 DOF manipulator,” in *Proc. 2nd Int. Conf. on Materials Science, Machinery and Energy Engineering* (*MSMEE 2017*), Atlantis Press, pp. 400–408, 2017. doi:10.2991/msmee-17.2017.81

[19] M. V. Bogdanovskyi, O. V. Pidtychenko and D. P. Gorbik, “Development of a software simulator for the study of robotic manipulator kinematics tasks and control” (in Ukrainian), *Technical Engineering* (Zhytomyr Polytechnic State University), no. 1(97), 2026.

[20] H. M. Al-Qahtani, A. A. Mohammed and M. Sunar, “Dynamics and control of a robotic arm having four links,” *Arabian Journal for Science and Engineering*, vol. 42, no. 5, pp. 1841–1852, 2017. doi:10.1007/s13369-016-2324-y

[21] X. Li, H. Sun, L. Liao and J. Song, “Simulation and comparison research of Lagrange and Kane dynamics modeling for the 4-DOF modular industrial robot,” in *Proc. 5th Int. Conf. on Advanced Design and Manufacturing Engineering* (*ICADME 2015*), Atlantis Press, pp. 251–254, 2015. doi:10.2991/icadme-15.2015.50

[22] N. G. Adar, H. Ören and R. Kozan, “Modelling and control of 5 dof robotic arm,” *Sakarya Üniversitesi Fen Bilimleri Enstitüsü Dergisi*, vol. 17, no. 1, 2013. doi:10.5505/saufbe.2013.07769

[23] J. Y. S. Luh, M. W. Walker and R. P. C. Paul, “On-line computational scheme for mechanical manipulators,” *Journal of Dynamic Systems, Measurement, and Control*, vol. 102, no. 2, pp. 69–76, 1980.

[24] M. W. Walker and D. E. Orin, “Efficient dynamic computer simulation of robotic mechanisms,” *Journal of Dynamic Systems, Measurement, and Control*, vol. 104, no. 3, pp. 205–211, 1982.

[25] R. Featherstone, *Rigid Body Dynamics Algorithms*. Springer, 2008.

[26] J. L. Mata-Machuca, R. Aguilar-López and J. Fonseca-Campos, “Mathematical modelling and simulation of a robot manipulator,” in *Frontiers in Mathematical Modelling Research*. Nova Science Publishers, pp. 149–174, 2022.

[27] P. R. Bonikila, R. K. Mandava and P. R. Vundavilli, “Development of path tracking control algorithm for a 4 DOF spatial manipulator using PID controller,” IGI Global, 2017; reprinted in *Robotic Systems: Concepts, Methodologies, Tools, and Applications*, 2020. doi:10.4018/978-1-7998-1754-3.ch023

[28] M. Takegaki and S. Arimoto, “A new feedback method for dynamic control of manipulators,” *Journal of Dynamic Systems, Measurement, and Control*, vol. 103, no. 2, pp. 119–125, 1981.

[29] R. Kelly, V. Santibáñez and A. Loría, *Control of Robot Manipulators in Joint Space*. Springer, 2005.

[30] J. Vidaković, V. Kvrgić, M. Lazarević and P. Stepanić, "Computed torque control for a spatial disorientation trainer," *Facta Universitatis, Series: Mechanical Engineering*, vol. 18, no. 2, pp. 269–280, 2020. doi:10.22190/FUME190919003V

[31] J. A. Shah and S. S. Rattan, "Dynamic analysis of two link robot manipulator for control design using PID computed torque control," *IAES Int. Journal of Robotics and Automation*, vol. 5, no. 4, pp. 277–283, 2016. doi:10.11591/ijra.v5i4.pp277-283

[32] R. Heradio, L. de la Torre and S. Dormido, "Virtual and remote labs in control education: A survey," *Annual Reviews in Control*, vol. 42, pp. 1–10, 2016. doi:10.1016/j.arcontrol.2016.08.001

[33] A. Gil, O. Reinoso, J. M. Marin, L. Paya and J. Ruiz, "Development and deployment of a new robotics toolbox for education," *Computer Applications in Engineering Education*, vol. 23, no. 3, pp. 443–454, 2015. doi:10.1002/cae.21615

[34] R. S. Othayoth, R. G. Chittawadigi, R. P. Joshi and S. K. Saha, "Robot kinematics made easy using RoboAnalyzer software," *Computer Applications in Engineering Education*, vol. 25, no. 5, pp. 669–680, 2017. doi:10.1002/cae.21828

[35] P. Corke and J. Haviland, "Not your grandmother's toolbox – the Robotics Toolbox reinvented for Python," in *Proc. IEEE Int. Conf. on Robotics and Automation* (*ICRA*), pp. 11357–11363, 2021.

[36] E. Rohmer, S. P. N. Singh and M. Freese, "V-REP: A versatile and scalable robot simulation framework," in *Proc. IEEE/RSJ Int. Conf. on Intelligent Robots and Systems* (*IROS*), pp. 1321–1326, 2013. doi:10.1109/IROS.2013.6696520

[37] S. Tselegkaridis and T. Sapounidis, "Simulators in educational robotics: A review," *Education Sciences*, vol. 11, no. 1, art. 11, 2021. doi:10.3390/educsci11010011

[38] S. A. Ajwad, R. U. Islam and J. Iqbal, "Exploring the training potential of recent virtual robotic platforms: A comprehensive review," *Journal of Advancements in Robotics*, vol. 4, no. 2, pp. 9–25, 2017.